



Islamic University of Technology

CSE 4508: RDBMS Lab

Project Report

RoomLagbe

A Student Housing Platform for IUT

Team HomeSick

- | | |
|---------------------------|-----------|
| 1. Rahatut Tahrim Mounota | 220041122 |
| 2. Nayeemul Hasan Prince | 220041125 |
| 3. Alfi Shahrin | 220041153 |

Course: RDBMS Lab (CSE 4508)
Lab Instructor: Mueeze Al Mushabbir
Department: Computer Science & Engineering
Institution: Islamic University of Technology
Date of Submission: 12 March 2026

Contents

1	Problem, Scope, and Project Overview	2
1.1	Problem Statement	2
1.2	Scope and Solution	2
1.3	Project Overview	2
2	List of Features	2
3	Tools and Technologies	3
3.1	Frontend	3
3.2	Backend	3
3.3	Database & Infrastructure	4
3.4	External APIs	4
3.5	Development Tools	4
4	Database Design	4
4.1	Schema Overview	4
4.2	ER Diagram	4
4.3	Entity and Table Relationships	5
4.4	Key Design Decisions	6
5	Procedural SQL, Advanced Queries, and Concepts	6
5.1	Database Triggers and Functions	6
5.1.1	Functions	6
5.1.2	Triggers	9
5.2	Advanced Queries	10
5.2.1	Wishlist: Get User Wishlist Details	10
5.2.2	Wishlist: Get Wishlist Counts Per Listing	11
5.2.3	Listing: Get Listing Details	11
5.2.4	Apartment Metrics: Wishlist Count Upsert	12
5.2.5	Geospatial Query: Nearby Location Search	12
5.2.6	Booking Conflict Detection	12
5.2.7	Admin Analytics Query	13
5.2.8	Database Performance Optimization	13
5.3	Other Database Concepts Used	13
6	Limitations	13
7	Modifications from Initial Scope	14
7.1	Features Removed or Descoped	14
7.2	Features Added	14
7.3	Schema Changes	14
8	Individual Contribution	15
9	GitHub Repository	16

1 Problem, Scope, and Project Overview

1.1 Problem Statement

University students in Bangladesh, especially at IUT, face major challenges finding affordable, trustworthy accommodation near campus. Existing solutions are fragmented (Facebook, WhatsApp, word-of-mouth), making it hard to compare options, verify landlords, or judge rent fairness. Owners also lack a structured way to reach students or manage interest.

1.2 Scope and Solution

Our project RoomLagbe is a web platform for the IUT student housing market. It enables:

- Owners to create/manage listings (with admin verification)
- Students to post room-shares
- Search/filter with interactive map and radius
- Structured visit requests with time-slot conflict prevention
- Identity verification and verified badges
- Analytics dashboards and areawise fair-rent scoring
- Commute calculator to campus

1.3 Project Overview

RoomLagbe is a full-stack app with React/Vite frontend, Node.js/Express backend, PostgreSQL (Supabase) database. It includes three user roles (Student, Owner, Admin) with distinct permissions. The schema is normalized (3NF), uses PostGIS for spatial data, and integrates external APIs for geocoding and routing.

2 List of Features

Feature	Description
User Authentication & Roles	JWT-based login/registration. Roles: Student, Owner, Admin. Role-based dashboards and route protection.
Identity Verification	Upload of student or owner ID/NID, admin approval, blue verification badge on listings.
Apartment Listing	Owners can post apartments, with full details on rent, amenities, availability and photos.
Room-Share Listing	Students can post spare rooms; system links to apartment and creates unified listing.
Amenity Selection	Multi-select amenities (Wi-Fi, AC, etc.) via normalized catalogue and junction tables.

Search & Filtering	Text, filter, and map-based search with radius filtering (Haversine formula).
Listing Detail Page	Full property info, photo gallery, amenities, owner info, fair rent score, visit request, commute widget.
Visit Request & Booking	Students request visits; 2-hour time-blocking prevents double-booking; status workflow.
Owner Visit Management	Owners approve/reject visit requests from dashboard.
Listing Lifecycle	Soft delete/close statuses; preserves analytics and audit trails.
Wishlist	Students can save listings; composite PK for idempotency; stats tracked.
Saved Searches	Students save filter presets as JSONB; re-apply with one click.
Commute Calculator	Calculates road-based distance/time to campus using OSRM API; fallback to Haversine.
Student Dashboard	Unified view: listings, requests, wishlists, saved searches, stats.
Owner Dashboard	Portfolio analytics: stats, per-listing breakdown, visit requests, analytics charts.
Admin Panel	Analytics: Verification queues, platform-wide stats, per-area analytics, top listings.
In-App Notifications	Bell icon, unread count, triggers for key events, partial index for fast queries.
Fair Rent Score	Compares price to area average; pre-calculated and stored for fast access.
Photo Upload	Multiple images per listing, Supabase storage, thumbnail selection, gallery view.

3 Tools and Technologies

3.1 Frontend

- React, Vite, Tailwind CSS
- React Router DOM, Axios
- React Leaflet, Recharts, Sonner, Lucide React

3.2 Backend

- Node.js, Express.js
- pg (node-postgres), jsonwebtoken, bcryptjs, multer
- @supabase/supabase-js, dotenv, cors

3.3 Database & Infrastructure

- PostgreSQL 15 (Supabase hosted), PostGIS, pgcrypto
- Supabase Storage for photos

3.4 External APIs

- OSM Nominatim (geocoding), OSRM (routing), OpenStreetMap tiles

3.5 Development Tools

- VS Code, Git/GitHub, Postman, dbdiagram.io

4 Database Design

4.1 Schema Overview

The RoomLagbe database consists of 16 tables, 5 ENUM types, 7 triggers, and 6 PL/pgSQL functions. It is normalized to 3NF, with intentional denormalization for performance. UUIDs are used as primary keys. PostGIS is used for spatial data and GiST indexes for proximity queries.

4.2 ER Diagram

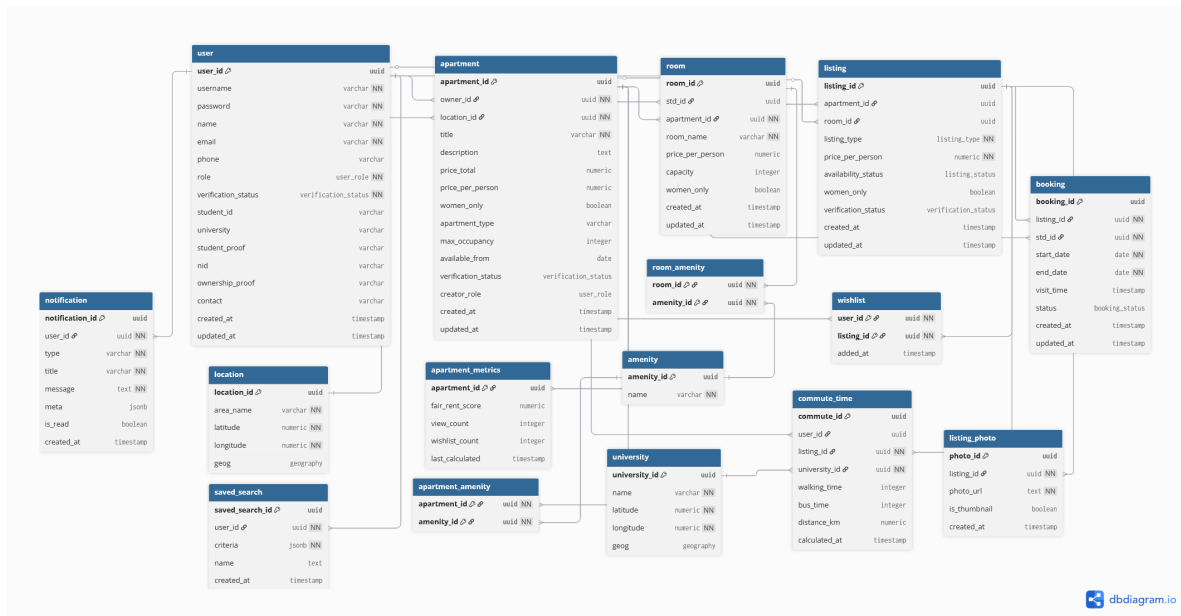


Figure 1: RoomLagbe Database ER Diagram

4.3 Entity and Table Relationships

The RoomLagbe schema is designed to model the real-world relationships between users, properties, locations, and platform features. Below is an overview of the key relationships:

- **User and Roles:** The `user` table stores all platform users, each with a unique role (Student, Owner, Admin). Users can own apartments, post room-shares, make bookings, receive notifications, and save listings to their wishlist.
- **Apartment and Owner:** Each `apartment` is owned by a user (`owner_id`), establishing a many-to-one relationship from apartments to users. An owner can have multiple apartments.
- **Apartment and Location:** Each apartment references a `location` (`location_id`), linking it to a specific geographic area. Locations store latitude, longitude, and a PostGIS geography point.
- **Room and Apartment:** The `room` table models individual rooms within apartments. Each room belongs to one apartment (`apartment_id`), and optionally to a student (`std_id`) if it is a room-share listing.
- **Listing Abstraction:** The `listing` table unifies both apartment and room listings. Each listing references either an apartment, a room, or both, and includes type, price, and status. This enables uniform search and filtering across all listings.
- **Amenities:** Amenities are managed via a normalized catalogue (`amenity` table). The `apartment_amenity` and `room_amenity` junction tables implement many-to-many relationships, allowing apartments and rooms to have multiple amenities.
- **Photos:** The `listing_photo` table stores multiple photos per listing, with a flag for the thumbnail image.
- **Bookings:** The `booking` table links students (`std_id`) to listings (`listing_id`), recording visit requests and booking status. Each booking references a user and a listing.
- **Wishlist:** The `wishlist` table is a composite-key junction table connecting users and listings, allowing users to save multiple listings and each listing to be saved by multiple users.
- **Saved Searches:** The `saved_search` table links users to their saved filter criteria, stored as JSONB.
- **Notifications:** The `notification` table stores messages for users, with a foreign key to the user table.
- **Apartment Metrics:** The `apartment_metrics` table stores analytics (fair rent score, views, wishlists) for each apartment, linked by `apartment_id`.
- **Commute Time:** The `commute_time` table links users, listings, and universities, storing commute metrics for each combination.
- **University:** The `university` table stores university locations, referenced by commute calculations.

- **Spatial Reference:** The `spatial_ref_sys` table is part of the PostGIS extension, supporting spatial operations.

Summary of Key Relationships:

- One-to-many: `user` → `apartment`, `apartment` → `room`, `apartment` → `apartment_metrics`, `user` → `booking`, `user` → `notification`
- Many-to-many: `apartment` ↔ `amenity`, `room` ↔ `amenity`, `user` ↔ `wishlist` ↔ `listing`
- Unifying abstraction: `listing` references both `apartment` and `room`, enabling flexible property representation
- Geospatial: `location` and `university` tables use PostGIS columns for spatial queries

This relational structure ensures data integrity, supports complex queries (e.g., search, analytics, geospatial filtering), and models the real-world interactions between students, owners, and properties on the RoomLagbe platform.

4.4 Key Design Decisions

- **Listing Abstraction:** Unified listing table for apartments and room-shares, enabling uniform search/filtering.
- **Soft Delete/Close:** Status enums instead of hard deletes, preserving analytics and audit trails.
- **Materialized Metrics:** Pre-calculated metrics (fair rent, views, wishlists) for fast access.
- **Spatial Data:** PostGIS geography columns and GiST indexes for future server-side radius search.
- **JSONB:** Saved search criteria stored as JSONB for flexibility.

5 Procedural SQL, Advanced Queries, and Concepts

5.1 Database Triggers and Functions

This section describes the PostgreSQL functions and triggers implemented in the database. These components automate data updates, maintain consistency, and support spatial queries using PostGIS.

5.1.1 Functions

update_modified_column() This trigger function automatically updates the `updated_at` column whenever a row is modified.

```

1 CREATE OR REPLACE FUNCTION update_modified_column()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.updated_at = now();
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
```

Listing 1: Update Modified Timestamp Function

location_set_geog() This function automatically generates a PostGIS geography point using latitude and longitude values.

```

1 CREATE OR REPLACE FUNCTION location_set_geog()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.geog := ST_SetSRID(ST_MakePoint(NEW.longitude, NEW.latitude),
5                               4326)::geography;
6     RETURN NEW;
7 END;
8 $$ LANGUAGE plpgsql;
```

Listing 2: Location Geography Auto-Population Function

university_set_geog() This function creates a geographic point for university records using their coordinates.

```

1 CREATE OR REPLACE FUNCTION university_set_geog()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.geog := ST_SetSRID(ST_MakePoint(NEW.longitude, NEW.latitude),
5                               4326)::geography;
6     RETURN NEW;
7 END;
8 $$ LANGUAGE plpgsql;
```

Listing 3: University Geography Auto-Population Function

get_listings_near_location(search_lat, search_lng, radius_meters) Returns listings located within a specified radius of given coordinates using PostGIS spatial queries.

```

1 CREATE OR REPLACE FUNCTION get_listings_near_location(
2     search_lat NUMERIC,
3     search_lng NUMERIC,
4     radius_meters INT DEFAULT 5000
5 )
6 RETURNS TABLE (
7     listing_id UUID,
8     distance_meters NUMERIC
9 ) AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT
13         l.listing_id,
14         ST_Distance(
15             loc.geog,
16             ST_SetSRID(ST_MakePoint(search_lng, search_lat), 4326)::geography
17         )::NUMERIC as distance_meters
18     FROM listing l
19     JOIN apartment apt ON l.apartment_id = apt.apartment_id
20     JOIN location loc ON apt.location_id = loc.location_id
```

```

21 WHERE ST_DWithin(
22     loc.geog,
23     ST_SetSRID(ST_MakePoint(search_lng, search_lat), 4326)::geography,
24     radius_meters
25 )
26 ORDER BY distance_meters;
27 END;
28 $$ LANGUAGE plpgsql;

```

Listing 4: Nearby Listing Search Function

calculate_listing_university_distance(p_listing_id, p_university_id) Calculates the geographic distance between a listing and a university.

```

1 CREATE OR REPLACE FUNCTION calculate_listing_university_distance(
2     p_listing_id UUID,
3     p_university_id UUID
4 )
5 RETURNS NUMERIC AS $$
6 DECLARE
7     distance_km NUMERIC;
8 BEGIN
9     SELECT
10         ST_Distance(loc.geog, uni.geog) / 1000.0
11     INTO distance_km
12     FROM listing l
13     JOIN apartment apt ON l.apartment_id = apt.apartment_id
14     JOIN location loc ON apt.location_id = loc.location_id
15     CROSS JOIN university uni
16     WHERE l.listing_id = p_listing_id
17         AND uni.university_id = p_university_id;
18
19     RETURN distance_km;
20 END;
21 $$ LANGUAGE plpgsql;

```

Listing 5: Listing-University Distance Calculation

create_user(__username, __password, __role) Creates a new user and returns the inserted record.

```

1 CREATE OR REPLACE FUNCTION create_user(
2     _username VARCHAR,
3     _password VARCHAR,
4     _role VARCHAR
5 )
6 RETURNS SETOF users AS $$
7 BEGIN
8     RETURN QUERY
9         INSERT INTO users (username, password, role)
10        VALUES (_username, _password, _role)
11        RETURNING *;
12 END;
13 $$ LANGUAGE plpgsql;

```

Listing 6: User Creation Function

5.1.2 Triggers

Triggers automatically execute functions when database events occur.

User Table Trigger

```
1 CREATE TRIGGER trg_user_updated_at
2 BEFORE UPDATE ON "user"
3 FOR EACH ROW
4 EXECUTE FUNCTION update_modified_column();
```

Listing 7: User Table Update Trigger

Location Table Trigger

```
1 DROP TRIGGER IF EXISTS trg_location_set_geog ON location;
2 CREATE TRIGGER trg_location_set_geog
3 BEFORE INSERT OR UPDATE OF latitude, longitude ON location
4 FOR EACH ROW
5 EXECUTE FUNCTION location_set_geog();
```

Listing 8: Location Geography Trigger

University Table Trigger

```
1 DROP TRIGGER IF EXISTS trg_university_set_geog ON university;
2 CREATE TRIGGER trg_university_set_geog
3 BEFORE INSERT OR UPDATE OF latitude, longitude ON university
4 FOR EACH ROW
5 EXECUTE FUNCTION university_set_geog();
```

Listing 9: University Geography Trigger

Apartment Table Trigger

```
1 DROP TRIGGER IF EXISTS trg_apartment_updated_at ON apartment;
2 CREATE TRIGGER trg_apartment_updated_at
3 BEFORE UPDATE ON apartment
4 FOR EACH ROW
5 EXECUTE FUNCTION update_modified_column();
```

Listing 10: Apartment Update Timestamp Trigger

Room Table Trigger

```
1 DROP TRIGGER IF EXISTS trg_room_updated_at ON room;
2 CREATE TRIGGER trg_room_updated_at
3 BEFORE UPDATE ON room
4 FOR EACH ROW
5 EXECUTE FUNCTION update_modified_column();
```

Listing 11: Room Update Timestamp Trigger

Listing Table Trigger

```
1 DROP TRIGGER IF EXISTS trg_listing_updated_at ON listing;
2 CREATE TRIGGER trg_listing_updated_at
3 BEFORE UPDATE ON listing
4 FOR EACH ROW
5 EXECUTE FUNCTION update_modified_column();
```

Listing 12: Listing Update Timestamp Trigger

Booking Table Trigger

```

1 DROP TRIGGER IF EXISTS trg_booking_updated_at ON booking;
2 CREATE TRIGGER trg_booking_updated_at
3 BEFORE UPDATE ON booking
4 FOR EACH ROW
5 EXECUTE FUNCTION update_modified_column();

```

Listing 13: Booking Update Timestamp Trigger

Users Table Trigger

```

1 CREATE TRIGGER update_user_modtime
2 BEFORE UPDATE ON users
3 FOR EACH ROW
4 EXECUTE FUNCTION update_modified_column();

```

Listing 14: Users Table Update Trigger

Listings Table Trigger

```

1 CREATE TRIGGER update_listing_modtime
2 BEFORE UPDATE ON listings
3 FOR EACH ROW
4 EXECUTE FUNCTION update_modified_column();

```

Listing 15: Listings Table Update Trigger

5.2 Advanced Queries

Several advanced SQL queries have been used for our system. These queries involve complex joins, aggregations, geospatial filtering using PostGIS, and optimized database operations.

5.2.1 Wishlist: Get User Wishlist Details

Retrieves detailed information about listings saved in a user's wishlist, including listing details, apartment or room information, location, owner information, and thumbnail image.

```

1 SELECT w.*,
2        l.listing_type, l.price_per_person, l.availability_status,
3        COALESCE(a.title, ra.title) as title,
4        COALESCE(a.title, ra.title) as apartment_title,
5        COALESCE(a.apartment_type, ra.apartment_type) as apartment_type,
6        r.room_name,
7        COALESCE(loc.area_name, rloc.area_name) as location,
8        COALESCE(u_owner.name, u_student.name) as owner_name,
9        lp.photo_url as thumbnail
10 FROM wishlist w
11 JOIN listing l ON w.listing_id = l.listing_id
12 LEFT JOIN apartment a ON l.apartment_id = a.apartment_id
13 LEFT JOIN room r ON l.room_id = r.room_id
14 LEFT JOIN apartment ra ON r.apartment_id = ra.apartment_id
15 LEFT JOIN location loc ON a.location_id = loc.location_id
16 LEFT JOIN location rloc ON ra.location_id = rloc.location_id
17 LEFT JOIN "user" u_owner ON (a.owner_id = u_owner.user_id)
18 LEFT JOIN "user" u_student ON (r.std_id = u_student.user_id)
19 LEFT JOIN listing_photo lp

```

```

20 ON l.listing_id = lp.listing_id AND lp.is_thumbnail = true
21 WHERE w.user_id = $1
22 ORDER BY w.added_at DESC;

```

Listing 16: User Wishlist Details Query

5.2.2 Wishlist: Get Wishlist Counts Per Listing

Counts how many users have added each listing to their wishlist.

```

1 SELECT l.listing_id,
2        COUNT(w.user_id) as wishlist_count,
3        l.listing_type,
4        a.title,
5        loc.area_name as location
6 FROM listing l
7 JOIN wishlist w ON l.listing_id = w.listing_id
8 LEFT JOIN apartment a ON l.apartment_id = a.apartment_id
9 LEFT JOIN location loc ON a.location_id = loc.location_id
10 GROUP BY l.listing_id, l.listing_type, a.title, loc.area_name;

```

Listing 17: Wishlist Count Aggregation

5.2.3 Listing: Get Listing Details

Retrieves comprehensive listing details including apartment, room, location, owner information, and thumbnail image.

```

1 SELECT l.*,
2        COALESCE(a.title, ra.title) as apartment_title,
3        r.room_name,
4        lp.photo_url as thumbnail,
5        COALESCE(loc.area_name, rloc.area_name) as location,
6        COALESCE(loc.latitude, rloc.latitude) as latitude,
7        COALESCE(loc.longitude, rloc.longitude) as longitude,
8        COALESCE(a.available_from, ra.available_from) as available_from,
9        COALESCE(a.apartment_type, ra.apartment_type) as apartment_type,
10       COALESCE(a.max_occupancy, ra.max_occupancy) as max_occupancy,
11       COALESCE(u_owner.name, u_student.name) as owner_name,
12       COALESCE(u_owner.phone, u_student.phone) as owner_phone,
13       COALESCE(u_owner.email, u_student.email) as owner_email
14 FROM listing l
15 LEFT JOIN apartment a ON l.apartment_id = a.apartment_id
16 LEFT JOIN room r ON l.room_id = r.room_id
17 LEFT JOIN apartment ra ON r.apartment_id = ra.apartment_id
18 LEFT JOIN location loc ON a.location_id = loc.location_id
19 LEFT JOIN location rloc ON ra.location_id = rloc.location_id
20 LEFT JOIN listing_photo lp
21 ON l.listing_id = lp.listing_id AND lp.is_thumbnail = true
22 LEFT JOIN "user" u_owner ON (a.owner_id = u_owner.user_id)
23 LEFT JOIN "user" u_student ON (r.std_id = u_student.user_id);

```

Listing 18: Detailed Listing Query

5.2.4 Apartment Metrics: Wishlist Count Upsert

Updates or inserts wishlist metrics for an apartment using PostgreSQL UPSERT functionality.

```
1 INSERT INTO apartment_metrics (apartment_id, wishlist_count)
2 VALUES ($1, (
3     SELECT COUNT(*) FROM wishlist w
4     JOIN listing l ON w.listing_id = l.listing_id
5     WHERE l.apartment_id = $1
6 ))
7 ON CONFLICT (apartment_id)
8 DO UPDATE SET
9     wishlist_count = (
10        SELECT COUNT(*) FROM wishlist w
11        JOIN listing l ON w.listing_id = l.listing_id
12        WHERE l.apartment_id = $1
13    );
```

Listing 19: Apartment Wishlist Metrics Upsert

5.2.5 Geospatial Query: Nearby Location Search

Uses PostGIS spatial functions to find locations within a small radius of given coordinates.

```
1 SELECT location_id, area_name
2 FROM location
3 WHERE ST_DWithin(
4     geog,
5     ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography,
6     50
7 )
8 LIMIT 1;
```

Listing 20: Nearby Location Detection

5.2.6 Booking Conflict Detection

Detects scheduling conflicts for booking visit times to avoid overlapping appointments.

```
1 SELECT b.booking_id
2 FROM booking b
3 WHERE b.listing_id = $1
4     AND b.status IN ('approved', 'pending')
5     AND b.visit_time IS NOT NULL
6     AND (
7         ($2 >= b.visit_time AND $2 < (b.visit_time + INTERVAL '2 hours'))
8         OR
9         (b.visit_time >= $2 AND b.visit_time < ($2::timestampz + INTERVAL
10             '2 hours'))
11     )
12 LIMIT 1;
```

Listing 21: Time-Conflict Detection for Bookings

5.2.7 Admin Analytics Query

Provides aggregated analytics for the administration dashboard using PostgreSQL FILTER clause.

```
1 SELECT
2   COUNT(*) AS total_listings,
3   COUNT(*) FILTER (WHERE listing_type = 'apartment') AS apartment_count
4   ,
5   COUNT(*) FILTER (WHERE listing_type = 'room') AS room_count
6 FROM listing;
```

Listing 22: Admin Listing Analytics

5.2.8 Database Performance Optimization

A partial index is used to improve performance when retrieving unread notifications.

```
1 CREATE INDEX idx_notification_unread
2 ON notification(user_id)
3 WHERE is_read = FALSE;
```

Listing 23: Partial Index for Unread Notifications

5.3 Other Database Concepts Used

- **PostGIS:** Spatial columns, GiST indexes, and triggers are used to support location-based queries for apartments and universities.
- **JSONB:** Flexible storage is used for dynamic user preferences and saved search filters.
- **UPSERT:** PostgreSQL ON CONFLICT ensures atomic updates of apartment metrics.
- **Haversine Formula:** Used on the client side for approximate radius filtering before executing precise spatial queries.

6 Limitations

- No server-side radius filtering (currently client-side Haversine is being used)
- No pagination in search results
- No real-time updates (no WebSocket/SSE)
- No in-app messaging
- No transaction wrapping for multi-table inserts
- Photo storage is Supabase-dependent
- Fair rent score requires dataset density
- Verification system is manual and dependent on admin

7 Modifications from Initial Scope

7.1 Features Removed or Descoped

- Real-time chat (removed; phone number shown instead)
- Server-side radius search (PostGIS infrastructure present, not yet deployed)
- PL/pgSQL business logic (moved to Node.js)

7.2 Features Added

- In-app notifications
- Soft delete/close statuses
- Booking cancellation by students
- Per-listing analytics
- Separate admin analytics page
- Fair rent score
- Commute calculator
- Verified badge

7.3 Schema Changes

- Added new tables: `notification`, `apartment_metrics`, `commute_time`, and `saved_search`.
- Extended enumerations: `listing_status` and `booking_status`.
- Added verification-related columns to the `user` table.

8 Individual Contribution

Team Member	Detailed Contributions
Rahatut Tahrim Mounota	• Initial and final database schema design (tables, relationships, normalization) • Complete ER diagram and documentation • Implementation of authentication (JWT, password hashing, role-based access) • Core backend models and REST API routes (user, listing, booking) • Admin dashboard: verification queues, analytics endpoints, and UI integration • Commute calculation logic and integration with OSRM API • SQL query optimization and review • Project documentation and report writing
Nayeemul Hasan Prince	• Finalization and migration of schema to Supabase/PostGIS • Implementation of all database triggers and PL/pgSQL functions • Data integrity enforcement (constraints, checks, foreign keys) • Owner dashboard: listing management, visit request workflow • Notification system: backend logic, trigger setup, and frontend integration • Visit scheduling logic (2-hour conflict prevention, booking status transitions) • Manual and Postman-based API testing • Contribution to project presentations and slides
Alfi Shahrin	• Advanced analytics SQL queries (fair rent, area stats, top listings) • Audit log design and implementation for rent changes • Booking system: student dashboard, request/cancel/approve flows • Fair rent score calculation and metrics table logic • Commute polyline visualization on frontend (Leaflet/OSRM) • Saved search and wishlist features (backend and frontend) • UI/UX improvements and bug fixes • Final report editing and formatting

9 GitHub Repository

URL: <https://github.com/Prince02031/RoomLagbe>

For setup and local development instructions, see the README in the repository.